

Regression

Regression

- A regression problem is one that identifies patterns and regularities in the mapping btwn i/p variables $\mathbf{x}^{(n)}$ and a corresponding continuous o/p variable $y^{(n)}$ from a training data set $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$
(Note $\mathbf{x}^{(n)} = [x_1^{(n)} \dots x_0^{(n)}]^T \in \mathbb{R}^D$ and $y_n \in \mathbb{R}$).
- A regression method typically returns a conditional distribution $p(y_n | \mathbf{x}_n)$ for the unknown y_n associated w/ a new test i/p $\mathbf{x}_n \rightarrow$ useful for predictions / interpretations (find correlation)
- 1 The characteristics of $p(y^{(n)} | \mathbf{x}^{(n)})$ should capture patterns observed in the data (e.g. changes in mean/variance)
- We use a model to encode, as function of some parameters θ , the type of patterns we expect to find in the data.

Linear regression model

- For the linear regression model, we assume that the relationship btwn i/p $\mathbf{x}^{(n)}$ and o/p $y^{(n)}$ in $p(y^{(n)} | \mathbf{x}^{(n)})$ is linear, w/ θ as parameters specifying this conditional distribution.
- For the 1D case, we have $\theta = \{\sigma^2, w_0, w_1\}$

$$y^{(n)} = w_0 + w_1 x_1^{(n)} + \epsilon^{(n)}, \quad \epsilon^{(n)} \sim N(0, \sigma^2)$$

$y^{(n)}$ is the sum of a const. and a Gaussian RV $\rightarrow y^{(n)}$ is Gaussian distributed.

$$E[y^{(n)}] = w_0 + w_1 x_1^{(n)} + E[\epsilon^{(n)}] = w_0 + w_1 x_1^{(n)} \quad \text{Var}[y^{(n)}] = E[(y^{(n)} - E[y^{(n)}])^2] = E[\epsilon^{(n)2}] = \sigma^2$$

$$\therefore p(y^{(n)} | \mathbf{x}^{(n)}, \theta) = N(y^{(n)} | w_0 + w_1 x_1^{(n)}, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left[-\frac{1}{2} \frac{(y^{(n)} - w_0 - w_1 x_1^{(n)})^2}{\sigma^2}\right]$$

- For higher dimensions, we have $\theta = \{\sigma^2, \mathbf{w} = [w_0 \dots w_D]^T\}$

$$y^{(n)} = w_0 + w_1 x_1^{(n)} + \dots + w_D x_D^{(n)} = [w_0 \dots w_D] \begin{bmatrix} 1 \\ x_1^{(n)} \\ \vdots \\ x_D^{(n)} \end{bmatrix} + \epsilon_n = \mathbf{w}^T \tilde{\mathbf{x}}^{(n)} + \epsilon_n, \quad \epsilon_n \sim N(0, \sigma^2)$$

where $\tilde{\mathbf{x}} = [1, x_1]^T$ is the augmented i/p

$y^{(n)}$ is the sum of a const. and a Gaussian RV $\rightarrow y^{(n)}$ is Gaussian distributed.

$$E[y^{(n)}] = \mathbf{w}^T \tilde{\mathbf{x}}^{(n)} + E[\epsilon^{(n)}] = \mathbf{w}^T \tilde{\mathbf{x}}^{(n)} \quad \text{Var}[y^{(n)}] = E[(y^{(n)} - E[y^{(n)}])^2] = E[\epsilon^{(n)2}] = \sigma^2$$

$$\therefore p(y^{(n)} | \mathbf{x}^{(n)}, \theta) = N(y^{(n)} | \mathbf{w}^T \tilde{\mathbf{x}}^{(n)}, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left[-\frac{1}{2} \frac{(y^{(n)} - \mathbf{w}^T \tilde{\mathbf{x}}^{(n)})^2}{\sigma^2}\right]$$

Maximum likelihood estimate (MLE).

- We find the MLE for θ by max. the likelihood function $p(y^{(1)} \dots y^{(N)} | \mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}, \theta)$ wrt θ .

$$\theta^{ML} = \underset{\theta}{\text{argmax}} p(y^{(1)} \dots y^{(N)} | \mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}, \theta)$$

- In practice, we max. the log-likelihood function $L(\theta) = \log p(y^{(1)} \dots y^{(N)} | \mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}, \theta)$

$\rightarrow$ we get the same sol'n since logarithm is monotonically increasing

$$\theta^{ML} = \underset{\theta}{\text{argmax}} p(y^{(1)} \dots y^{(N)} | \mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}, \theta) = \underset{\theta}{\text{argmax}} L(\theta)$$

$\rightarrow$ taking the log can prevent overflow / underflow problems when computing products.

For Personal Use Only -bkwk2

- Assuming that each data pt is independent, then

$$p(\mathbf{y} | \mathbf{X}, \theta) = p(y^{(1)} \dots y^{(N)} | x^{(1)} \dots x^{(N)}, \theta) = \prod_{n=1}^N p(y^{(n)} | x^{(n)}, \theta) = \prod_{n=1}^N N(y^{(n)} | \mathbf{w}^T \mathbf{x}^{(n)}, \sigma^2)$$

the loglikelihood $L(\theta)$ is then given by

$$L(\theta) = \log p(y^{(1)} \dots y^{(N)} | x^{(1)} \dots x^{(N)}, \theta) = \sum_{n=1}^N \log p(y^{(n)} | x^{(n)}, \theta) = \sum_{n=1}^N \log N(y^{(n)} | \mathbf{w}^T \mathbf{x}^{(n)}, \sigma^2)$$

$$= \sum_{n=1}^N \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (y^{(n)} - \mathbf{w}^T \mathbf{x}^{(n)})^2 \right] = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w})^T (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w})$$

$$L(\theta) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{w}^T \tilde{\mathbf{X}} \tilde{\mathbf{X}}^T \mathbf{w} - 2\mathbf{y}^T \tilde{\mathbf{X}} \mathbf{w} + \mathbf{y}^T \mathbf{y})$$

where $\mathbf{y} = [y_1 \dots y_N]^T$ and $\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{x}^{(1)} \\ \vdots \\ \mathbf{x}^{(N)} \end{bmatrix}$

- Differentiating $L(\theta)$ wrt $\mathbf{w}$,

$$\frac{\partial L(\theta)}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} \left(-\frac{1}{2\sigma^2} (\mathbf{w}^T \tilde{\mathbf{X}} \tilde{\mathbf{X}}^T \mathbf{w} - 2\mathbf{y}^T \tilde{\mathbf{X}} \mathbf{w}) \right) = -\frac{\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \mathbf{w}}{\sigma^2} + \frac{\tilde{\mathbf{X}}^T \mathbf{y}}{\sigma^2} = 0$$

$$\therefore \mathbf{w} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \mathbf{y}$$

Differentiating $L(\theta)$ wrt σ^2 ,

$$\frac{\partial L(\theta)}{\partial \sigma^2} = \frac{\partial}{\partial \sigma^2} \left(-\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w})^T (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w}) \right) = -\frac{N}{2\sigma^2} + \frac{1}{2\sigma^4} (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w})^T (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w}) = 0$$

$$\therefore \sigma^2 = \frac{1}{N} (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w})^T (\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w})$$

+ when $N \leq D$, the MLE $\mathbf{w} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \mathbf{y}$ is not defined, as $\tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$ is not invertible

→ we have many values of $\mathbf{w}$ that fit the training data equally well w/ zero error.

Non-linear (basis function) regression

- Linear regression can model non-linear relationships by replacing $\mathbf{x}^{(n)}$ w some non-linear function of the i/ps $\phi(\mathbf{x}^{(n)}) = [\phi_1(\mathbf{x}^{(n)}) \dots \phi_M(\mathbf{x}^{(n)})]^T$ (so $\mathbf{w}^T \mathbf{x}^{(n)} \rightarrow \mathbf{w}^T \phi(\mathbf{x}^{(n)})$)

- Inference does not change, we just replace each $\mathbf{x}^{(n)}$ w the new $\phi(\mathbf{x}^{(n)})$

- Common basis functions $\phi_n(\mathbf{x})$ include:

↳ polynomial for 1D data: $\phi_n(\mathbf{x}^{(n)}) = x^{(n)m}$

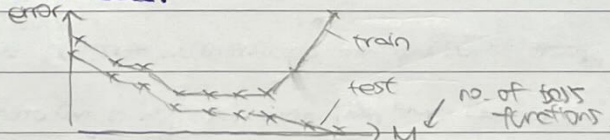
↳ Gaussian radial basis functions: $\phi_n(\mathbf{x}^{(n)}) = \exp\left[-\frac{(\mathbf{x}^{(n)} - \mathbf{c}_n)^T (\mathbf{x}^{(n)} - \mathbf{c}_n)}{2s^2}\right]$

↳ sigmoidal basis functions: $\phi_n(\mathbf{x}^{(n)}) = \sigma\left(\frac{(\mathbf{x}^{(n)} - \mathbf{c}_n)^T (\mathbf{x}^{(n)} - \mathbf{c}_n)}{s^2}\right)$, $\sigma(x) = \frac{1}{1 + \exp(-x)}$

+ Linear in terms of transformed i/ps $\phi(\mathbf{x}^{(n)})$ but non-linear in terms of original i/ps $\mathbf{x}^{(n)}$.

overfitting.

- A large no. of basis functions can lead to overfitting - the model fits the training data well but it performs poorly on new test data.



- As M increases, the magnitude of the entries of the MLE of $\mathbf{w}$ become large → large oscillations in the model's predictions → solve this by using a prior to favor the entries of $\mathbf{w}$ to be small.

Bayesian linear regression

Multivariate Gaussian distribution

- The density of a D -dimensional vector $\underline{x} \in \mathbb{R}^D$ is

$$N(\underline{x} | \underline{m}, \underline{\Sigma}) = \frac{1}{\sqrt{(2\pi)^D |\underline{\Sigma}|}} \exp\left[-\frac{1}{2}(\underline{x} - \underline{m})^T \underline{\Sigma}^{-1}(\underline{x} - \underline{m})\right] \propto \exp\left[-\frac{1}{2}\underline{x}^T \underline{\Sigma}^{-1} \underline{x} + \underline{m}^T \underline{\Sigma}^{-1} \underline{x}\right]$$

i.e. the density is prop. to the exponential of a quadratic function of $\underline{x}$, w/ normalisation const. Z

$$Z = \sqrt{(2\pi)^D |\underline{\Sigma}|} \exp\left(\frac{1}{2} \underline{m}^T \underline{\Sigma}^{-1} \underline{m}\right)$$

where $\underline{m} \in \mathbb{R}^D$ is the mean vector and $\underline{\Sigma} \in \mathbb{R}^{D \times D}$ is the covariance matrix

- The general gaussian $p(\underline{x}) = N(\underline{x} | \underline{m}, \underline{\Sigma})$ can be obtained from the standard gaussian $p(\underline{z}) = N(\underline{z} | \underline{0}, \underline{I})$

$$\underline{x} = \underline{m} + \underline{\Sigma}^{1/2} \underline{z}$$

where $\underline{\Sigma}^{1/2}$ is the matrix square root, defined as $\underline{\Sigma} = \underline{\Sigma}^{1/2} (\underline{\Sigma}^{1/2})^T$

cholesky decomposition

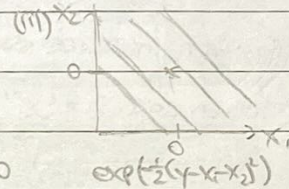
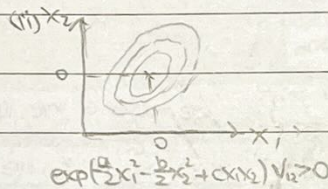
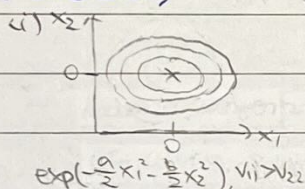
Effect of parameters on the density function

- For the general gaussian density,

$$p(\underline{x}) = N(\underline{x} | \underline{m}, \underline{\Sigma}) = \frac{1}{\sqrt{(2\pi)^D |\underline{\Sigma}|}} \exp\left[-\frac{1}{2}(\underline{x} - \underline{m})^T \underline{\Sigma}^{-1}(\underline{x} - \underline{m})\right]$$

the parameter $\underline{m}$ determines the mode (location) and $\underline{\Sigma}$ scales + rotates the space.

- For the $D=2$ case,



$$(i) \exp\left(-\frac{1}{2}x_1^2 - \frac{1}{2}x_2^2\right) = c \rightarrow ax_1^2 + bx_2^2 = k$$

$$(ii) \exp\left(-\frac{1}{2}x_1^2 - \frac{1}{2}x_2^2 + c x_1 x_2\right) = c \rightarrow ax_1^2 + bx_2^2 - 2c x_1 x_2 = k$$

$$(iii) \exp\left(-\frac{1}{2}(y - x_1 - x_2)^2\right) = c \rightarrow x_1 + x_2 = y + k$$

Linear combination of Gaussian RVs

- Let $p(\underline{x}) = N(\underline{x} | \underline{0}, \underline{V}_1)$ and $p(\underline{e}) = N(\underline{e} | \underline{0}, \underline{V}_2)$. Consider the transformation

$$\underline{y} = \underline{W} \underline{x} + \underline{e}$$

As linear combinations of independent Gaussian RVs are Gaussian, $p(\underline{y})$ is Gaussian, $p(\underline{y}) = N(\underline{y} | \underline{m}_3, \underline{V}_3)$

$$\hookrightarrow \underline{m}_3 = E[\underline{y}] = \underline{W} E[\underline{x}] + E[\underline{e}] = \underline{0} \rightarrow \underline{m}_3 = \underline{0}$$

$$\begin{aligned} \hookrightarrow \underline{V}_3 &= E[\underline{y} \underline{y}^T] - E[\underline{y}] E[\underline{y}]^T = E[(\underline{W} \underline{x} + \underline{e})(\underline{W} \underline{x} + \underline{e})^T] = E[\underline{W} \underline{x} \underline{x}^T \underline{W}^T + \underline{e} \underline{x}^T \underline{W}^T + \underline{W} \underline{x} \underline{e}^T + \underline{e} \underline{e}^T] \\ &= \underline{W} E[\underline{x} \underline{x}^T] \underline{W}^T + E[\underline{e}] E[\underline{x}]^T \underline{W}^T + \underline{W} E[\underline{x}] E[\underline{e}]^T + E[\underline{e} \underline{e}^T] = \underline{W} \underline{V}_1 \underline{W}^T + \underline{V}_2 \rightarrow \underline{V}_3 = \underline{W} \underline{V}_1 \underline{W}^T + \underline{V}_2 \end{aligned}$$

⚡ Note that $\underline{x}$ and $\underline{e}$ are assumed to be independent, so $E[\underline{x} \underline{e}^T] = E[\underline{x}] E[\underline{e}]^T$ etc.

Product of Gaussian densities

- Let $p(x) = N(x|m_1, \Sigma_1)$ and $q(x) = N(x|m_2, \Sigma_2)$, consider the product

$$t(x) \propto p(x)q(x)$$

since the product of exponentials of quadratic functions is also the exponential of a quadratic function, $t(x)$ is also Gaussian, $t(x) = N(x|m_3, \Sigma_3)$

$$t(x) \propto p(x)q(x) = N(x|m_1, \Sigma_1)N(x|m_2, \Sigma_2) = \frac{1}{(2\pi)^{D/2}|\Sigma_1|^{1/2}} \frac{1}{(2\pi)^{D/2}|\Sigma_2|^{1/2}} \exp\left[-\frac{1}{2}(x-m_1)^T \Sigma_1^{-1}(x-m_1) - \frac{1}{2}(x-m_2)^T \Sigma_2^{-1}(x-m_2)\right]$$

$$\propto \exp\left[-\frac{1}{2}x^T(\Sigma_1^{-1} + \Sigma_2^{-1})x + (m_1^T \Sigma_1^{-1} + m_2^T \Sigma_2^{-1})x\right]$$

Noting that $t(x) = N(x|m_3, \Sigma_3) \propto \exp\left(-\frac{1}{2}x^T \Sigma_3^{-1}x + m_3^T \Sigma_3^{-1}x\right)$,

"natural parameters"
→ empty sum

$$\Sigma_3^{-1} = (\Sigma_1^{-1} + \Sigma_2^{-1})$$

$$\rightarrow \Sigma_3 = (\Sigma_1^{-1} + \Sigma_2^{-1})^{-1}$$

$$m_3^T \Sigma_3^{-1} = m_1^T \Sigma_1^{-1} + m_2^T \Sigma_2^{-1}$$

$$\rightarrow m_3 = \Sigma_3 (m_1^T \Sigma_1^{-1} + m_2^T \Sigma_2^{-1})^T$$

completing the square.

- Let $p(x) = N(x|m, \Sigma) \propto \exp\left[-\frac{1}{2}x^T \Sigma^{-1}x + m^T \Sigma^{-1}x\right]$ and we are given an exponential of a quadratic function $q(x) \propto \exp\left[-\frac{1}{2}x^T \underline{B}x + \underline{b}^T x\right]$ to match w/.

$$\underline{B} = \Sigma^{-1} \rightarrow \underline{\Sigma} = \underline{B}^{-1}$$

$$\underline{b}^T = m^T \Sigma^{-1} \rightarrow$$

$$\underline{m} = \underline{\Sigma} \underline{b}$$

Bayesian inference

- using a large no. of basis functions can lead to overfitting - the model fits the training data well but performs poorly on new test data $\rightarrow \|w^M\|$ becomes large $\rightarrow$ large oscillations in model predictions.
- we can favour smooth solns using priors to enforce $\underline{\theta}$ to be small.

- the posterior distribution for $\underline{\theta}$, $p(\underline{\theta}|\underline{y}, \underline{X})$ is obtained by Bayes rule

$$p(\underline{\theta}|\underline{y}, \underline{X}) = \frac{p(\underline{y}|\underline{X}, \underline{\theta})p(\underline{\theta}|\underline{X})}{p(\underline{y}|\underline{X})}$$

where $p(\underline{y}|\underline{X}, \underline{\theta}) = \prod_{n=1}^N p(y^{(n)}|\underline{x}^{(n)}, \underline{\theta})$ is the likelihood, $p(\underline{y}|\underline{X})$ is the model evidence and $p(\underline{\theta}|\underline{X}) = p(\underline{\theta})$ is the prior distribution.

- the predictive distribution for y_* given a new corresponding x_* is

$$p(y_*|\underline{x}_*, \underline{y}, \underline{X}) = \int p(y_*, \underline{\theta}|\underline{x}_*, \underline{y}, \underline{X}) d\underline{w} = \int p(y_*|\underline{x}_*, \underline{\theta}) p(\underline{\theta}|\underline{y}, \underline{X}) d\underline{\theta}$$

i.e. weighted average of the predictions $p(y_*|\underline{x}_*, \underline{\theta})$, over the posterior distribution $p(\underline{\theta}|\underline{y}, \underline{X})$.

- Exact inference is possible if prior and noise distributions are Gaussian.

Bayesian linear regression

- Given data $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, we assume the linear regression model

$$y^{(n)} = \mathbf{w}^T \mathbf{x}^{(n)} + \epsilon_n, \quad \epsilon_n \sim N(0, \sigma^2)$$

to simplify inference, assume σ^2 is known - the only unknown is $\mathbf{w}$.

- choose the prior for $\mathbf{w}$ to be a zero-mean isotropic Gaussian,

$$p(\mathbf{w}) = N(\mathbf{w} | 0, \lambda^{-1} \mathbf{I}) \propto \exp\left[-\frac{1}{2} \mathbf{w}^T \lambda \mathbf{I} \mathbf{w}\right]$$

Recall that the likelihood under Gaussian noise is

$$p(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N N(y^{(n)} | \mathbf{x}^{(n)T} \mathbf{w}, \sigma^2) \propto \exp\left[-\frac{\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}}{2\sigma^2} + \frac{\mathbf{y}^T \mathbf{X} \mathbf{w}}{\sigma^2}\right]$$

The posterior density is the product of two Gaussian densities $\rightarrow$ posterior is Gaussian $N(\mathbf{m}, \mathbf{V})$

$$p(\mathbf{w} | \mathbf{y}, \mathbf{X}, \sigma^2) \propto p(\mathbf{y} | \mathbf{X}, \mathbf{w}) p(\mathbf{w}) \propto \exp\left[-\frac{1}{2} \mathbf{w}^T \left(\frac{\mathbf{X}^T \mathbf{X}}{\sigma^2} + \lambda \mathbf{I}\right) \mathbf{w} + \frac{\mathbf{y}^T \mathbf{X}}{\sigma^2} \mathbf{w}\right]$$

therefore $\mathbf{V} = \left(\frac{\mathbf{X}^T \mathbf{X}}{\sigma^2} + \lambda \mathbf{I}\right)^{-1}$, $\mathbf{m} = \mathbf{V} \frac{\mathbf{X}^T \mathbf{y}}{\sigma^2}$

(Alternatively, note $\mathbf{V}^{-1} = \frac{\mathbf{X}^T \mathbf{X}}{\sigma^2} + (\lambda \mathbf{I})^{-1}$, $\mathbf{m}^T \mathbf{V}^{-1} = \frac{\mathbf{y}^T \mathbf{X}}{\sigma^2} + \mathbf{0}^T (\lambda \mathbf{I})^{-1}$)

- As $\lambda \rightarrow 0$ (prior is flat), $\mathbf{w}^{\text{MAP}} = \mathbf{m} = \mathbf{w}^{\text{ML}}$ i.e. ML and MAP solns are identical.

Bayesian predictive distribution

- The predictive distribution for y_* given a new corresponding $\mathbf{x}_*$ is

$$p(y_* | \mathbf{x}_*, \mathbf{y}, \mathbf{X}) = \int p(y_* | \mathbf{x}_*, \mathbf{w}) p(\mathbf{w} | \mathbf{y}, \mathbf{X}) d\mathbf{w} = \int N(y_* | \mathbf{x}_*^T \mathbf{w}, \sigma^2) N(\mathbf{w} | \mathbf{m}, \mathbf{V}) d\mathbf{w}$$

- Instead of directly computing the integral, we note $y_* = \mathbf{w}^T \mathbf{x}_* + \epsilon_*$, where $\mathbf{w} \sim N(\mathbf{m}, \mathbf{V})$, $\epsilon_* \sim N(0, \sigma^2)$,

$$p(y_* | \mathbf{x}_*, \mathbf{y}, \mathbf{X}) = N(y_* | m_*, v_*)$$

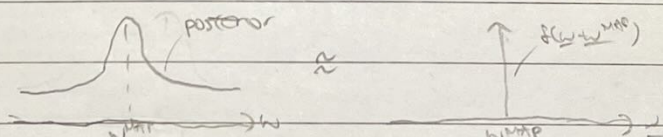
where $m_* = E[y_*] = E[\mathbf{w}^T \mathbf{x}_*] + E[\epsilon_*] = \mathbf{m}^T \mathbf{x}_* \rightarrow m_* = \mathbf{m}^T \mathbf{x}_*$

$$v_* = \text{Var}[y_*] = \text{Var}[\mathbf{w}^T \mathbf{x}_*] + \text{Var}[\epsilon_*] = \mathbf{x}_*^T \mathbf{V} \mathbf{x}_* + \sigma^2 \rightarrow v_* = \mathbf{x}_*^T \mathbf{V} \mathbf{x}_* + \sigma^2$$

- We can reduce overfitting and obtain confidence bounds $m_* \pm 1/2 v_*$ in our predictions.

Maximum a priori (MAP) inference

- Assume that the posterior is well approximated by a pt mass at its mode



The predictive distribution becomes

$$p(y_* | \mathbf{x}_*, \mathbf{y}, \mathbf{X}) = \int p(y_* | \mathbf{x}_*, \mathbf{w}) p(\mathbf{w} | \mathbf{y}, \mathbf{X}) d\mathbf{w} \approx \int p(y_* | \mathbf{x}_*, \mathbf{w}) \delta(\mathbf{w} - \mathbf{w}^{\text{MAP}}) d\mathbf{w} = p(y_* | \mathbf{x}_*, \mathbf{w}^{\text{MAP}})$$

- MAP inference fails to generate confidence bounds in the resulting predictions.

- The MAP soln $\mathbf{w}^{\text{MAP}}$ is obtained as follows

$$\mathbf{w}^{\text{MAP}} = \underset{\mathbf{w}}{\text{argmax}} p(\mathbf{w} | \mathbf{y}, \mathbf{X}) = \underset{\mathbf{w}}{\text{argmax}} p(\mathbf{y} | \mathbf{w}, \mathbf{X}) p(\mathbf{w}) = \underset{\mathbf{w}}{\text{argmax}} [\log p(\mathbf{y} | \mathbf{w}, \mathbf{X}) + \log p(\mathbf{w})]$$

For $p(\mathbf{w}) = N(\mathbf{w} | 0, \lambda^{-1} \mathbf{I})$, $\mathbf{w}^{\text{MAP}} = \underset{\mathbf{w}}{\text{argmax}} [\log p(\mathbf{y} | \mathbf{w}, \mathbf{X}) - \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}] = \left(\frac{\mathbf{X}^T \mathbf{X}}{\sigma^2} + \lambda \mathbf{I}\right)^{-1} \frac{\mathbf{X}^T \mathbf{y}}{\sigma^2}$

For Personal Use Only -bkwk2

Classification

Classification

Classification

- As in regression, we are given a training set $D = \{(\underline{x}^{(n)}, y^{(n)})\}_{n=1}^N$ consisting of N i/p $\underline{x}^{(n)} = [x_1^{(n)} \dots x_D^{(n)}]^T$ and o/p $y^{(n)}$, but the o/p's are now discrete: $y^{(n)} \in \{1, 2, \dots, C\}$ where C is the no. of classes/categories.
- In classification, we aim to identify a partition of the i/p space into C decision regions, one for each class. Each new i/p is assigned the class of its corresponding decision region, along w/ a measure of confidence.
- The decision regions are separated by decision boundaries — these are pts at which two classes have equal predictive probability.

Problems with methods for regression

- A way of using methods for regression for classification is to let the o/p $\underline{y}^{(n)}$ be a C -dimensional vector w/ one-hot encoding of the class for $\underline{x}^{(n)}$ (if the class is c , $y_i^{(n)} = 1 \{i=c\}$).
- We can then solve C linear regression problems, one for each class,

$$\underline{w} = (\underline{X}^T \underline{X})^{-1} \underline{X}^T \underline{y}$$

where $\underline{y} = \begin{bmatrix} y_1^{(1)} \\ \vdots \\ y_1^{(N)} \end{bmatrix}$ and we predict the class w/ the highest entry of $\underline{\tilde{x}}^T \underline{w}$.

- For the binary case, we could set a simple threshold to determine the class (positive class label if the o/p of the regression model is above the threshold and vice versa).
- These methods have issues of underrepresenting certain classes or varying the threshold (underfitting hypothesis) as we collect more data pts.

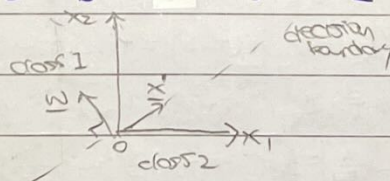
Deterministic linear classification

- Deterministic linear classification works by mapping the o/p of the linear model into discrete class labels. For the binary case, assume $y^{(n)} \in \{0, 1\}$, then we can define

$$y^{(n)} = H(\underline{w}^T \underline{\tilde{x}}^{(n)})$$

where $H(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$ is the Heaviside step function.

- The decision boundary is where $\underline{w}^T \underline{\tilde{x}}^{(n)} = 0 \rightarrow \underline{w}$ is orthogonal to $\underline{\tilde{x}}^{(n)}$



- This model gives deterministic (hard) predictions — misclassification errors are not allowed and inference is hard (cannot use gradient-based methods to find the MLE)

↑
step function is
not differentiable

Probabilistic linear classification

- Probabilistic linear classification works by mapping the output of the linear model into class probabilities (between 0 and 1). For the binary case, we define

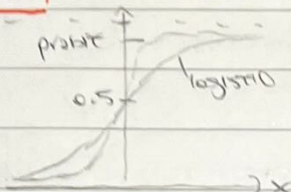
$$p(y^{(n)} = 1 | \mathbf{x}, \mathbf{w}) = f(\mathbf{w}^T \mathbf{x})$$

where $f(x)$ is a monotonically increasing function that maps $\mathbb{R} \rightarrow [0, 1]$.

- Typical functions $f(x)$ include:

→ Logistic function: $\sigma(x) = \frac{1}{1+e^{-x}}$

→ Probit function: $\Phi(x) = \int_{-\infty}^x \mathcal{N}(z|0,1) dz$



• Note that the functions are symmetric about (0, 0.5), so $f(-x) = 1 - f(x)$

- The likelihood function now allows for misclassification errors w/ some probability, and has a non-zero gradient that can be used to find the MLE.

Logistic regression.

- In logistic regression, we use $f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$.

(i) $y^{(n)} \in \{-1, 1\}$

$$p(y^{(n)} | \mathbf{x}^{(n)}, \mathbf{w}) = \frac{1+y^{(n)}}{2} \sigma(\mathbf{w}^T \mathbf{x}^{(n)}) + \frac{1-y^{(n)}}{2} (1 - \sigma(\mathbf{w}^T \mathbf{x}^{(n)})) = \sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)})$$

$$L(\mathbf{w}) = \log p(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \sum_{n=1}^N \log p(y^{(n)} | \mathbf{x}^{(n)}, \mathbf{w}) = \sum_{n=1}^N \log \sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)})$$

Noting that $\frac{d\sigma(x)}{dx} = \sigma(x)(1-\sigma(x)) = \sigma(x)\sigma(-x)$,

$$\frac{dL(\mathbf{w})}{d\mathbf{w}} = \frac{d}{d\mathbf{w}} \sum_{n=1}^N \log \sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)}) = \sum_{n=1}^N \frac{d}{d\mathbf{w}} \log \sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)})$$

$$= \sum_{n=1}^N \frac{1}{\sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)})} \sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)}) \sigma(-y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)}) \cdot y^{(n)} \mathbf{x}^{(n)}$$

$$\frac{dL(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N y^{(n)} (1 - \sigma(y^{(n)} \mathbf{w}^T \mathbf{x}^{(n)})) \mathbf{x}^{(n)}$$

(ii) $y^{(n)} \in \{0, 1\}$

$$p(y^{(n)} | \mathbf{x}^{(n)}, \mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{x}^{(n)})^{y^{(n)}} (1 - \sigma(\mathbf{w}^T \mathbf{x}^{(n)}))^{1-y^{(n)}}$$

$$L(\mathbf{w}) = \log p(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \sum_{n=1}^N \log p(y^{(n)} | \mathbf{x}^{(n)}, \mathbf{w}) = \sum_{n=1}^N [y^{(n)} \log \sigma(\mathbf{w}^T \mathbf{x}^{(n)}) + (1-y^{(n)}) \log \sigma(-\mathbf{w}^T \mathbf{x}^{(n)})]$$

Noting that $\frac{d\sigma(x)}{dx} = \sigma(x)(1-\sigma(x)) = \sigma(x)\sigma(-x)$,

$$\frac{dL(\mathbf{w})}{d\mathbf{w}} = \frac{d}{d\mathbf{w}} \sum_{n=1}^N [y^{(n)} \log \sigma(\mathbf{w}^T \mathbf{x}^{(n)}) + (1-y^{(n)}) \log \sigma(-\mathbf{w}^T \mathbf{x}^{(n)})]$$

$$= \sum_{n=1}^N \left[\frac{d}{d\mathbf{w}} (y^{(n)} \log \sigma(\mathbf{w}^T \mathbf{x}^{(n)})) + \frac{d}{d\mathbf{w}} ((1-y^{(n)}) \log \sigma(-\mathbf{w}^T \mathbf{x}^{(n)})) \right]$$

$$= \sum_{n=1}^N \left[y^{(n)} \frac{\sigma(\mathbf{w}^T \mathbf{x}^{(n)}) \sigma(-\mathbf{w}^T \mathbf{x}^{(n)})}{\sigma(\mathbf{w}^T \mathbf{x}^{(n)})} \mathbf{x}^{(n)} + (1-y^{(n)}) \frac{\sigma(-\mathbf{w}^T \mathbf{x}^{(n)}) \sigma(\mathbf{w}^T \mathbf{x}^{(n)})}{\sigma(-\mathbf{w}^T \mathbf{x}^{(n)})} (-\mathbf{x}^{(n)}) \right]$$

$$\frac{dL(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y^{(n)} - \sigma(\mathbf{w}^T \mathbf{x}^{(n)})) \mathbf{x}^{(n)}$$

- There is no closed form sol'n for the MLE → we numerical methods (gradient ascent)

For Personal Use Only -bkwk2

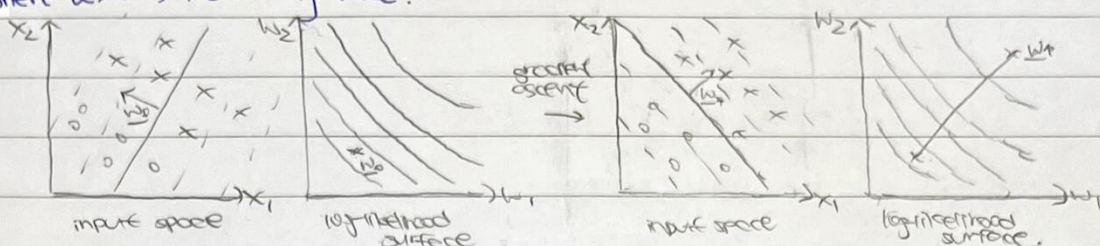
Gradient ascent

- The batch gradient ascent rule to maximise $L(w)$ is

$$w^{new} \leftarrow w^{old} + \alpha \frac{dL(w^{old})}{dw}$$

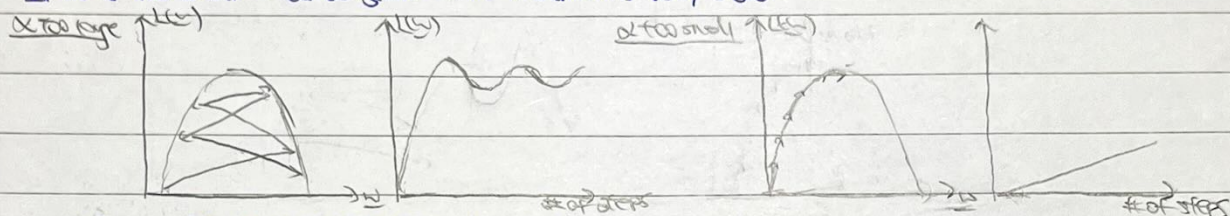
where $\alpha > 0$ is the learning rate.

depends on $y^{(n)} \in \{-1, 1\}$ vs $y^{(n)} \in \{0, 1\}$.



- If α is too large $\rightarrow$ optimisation bounces around the maximum, and could even diverge.

If α is too small $\rightarrow$ convergence to the maximum is very slow



There is no rule to choose α optimally $\rightarrow$ find α by trial and error.

Linear classification with more than 2 classes.

- We can map multiple dpts to discrete class labels using the max. function.

$$y^{(n)} = \underset{k \in \{1, \dots, C\}}{\operatorname{argmax}} w_k^T x$$

but this has similar problems as in the deterministic binary classification case

- We can instead use the softmax function to map the dpts into class probabilities

equivalent to $p(y^{(n)} | x^{(n)})$ for logistic regression

$$p(y^{(n)} = k | w_1, \dots, w_C, x^{(n)}) = \frac{\exp(w_k^T x^{(n)})}{\sum_{i=1}^C \exp(w_i^T x^{(n)})}$$

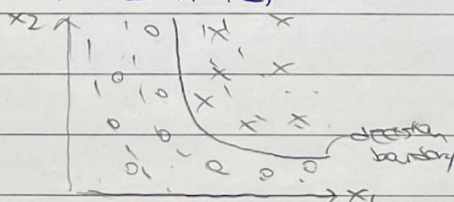
Note this is equivalent to logistic regression when $C=2$ (softmax is a generalisation of linear regression)

Non-linear logistic regression.

- We can replace x w/ non-linear functions of the inputs $\phi(x) = [\phi_1(x) \dots \phi_M(x)]^T$ to convert from linear to non-linear logistic regression.

- Non-linear logistic regression gives non-linear decision boundaries.

* Inference does not change, we just replace $x^{(n)}$ w/ $\phi(x^{(n)})$



Dimensionality reduction and PCA

Dimensionality reduction

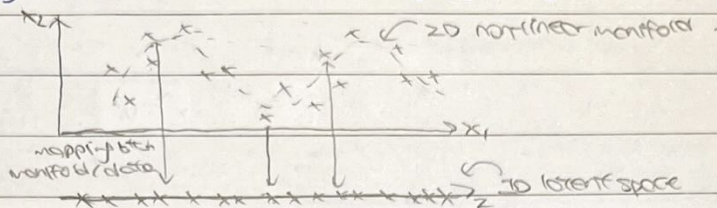
- Dimensionality reduction is a type of problem in machine learning requiring to find

→ A low dimensional representation (manifold) of data

→ A mapping from data to manifold coordinates and back.

- In general, the resulting manifold is highly nonlinear, but it can be unfolded and mapped non-linearly into a latent plane

- eg: A 2D manifold in 2D space is unfolded into a latent real line

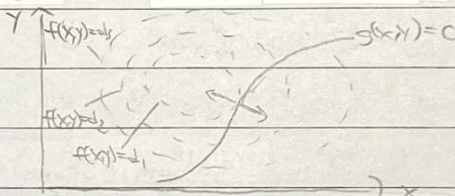


Interpretation of the pts stays in the data manifold in the 2D latent space, but not in the 2D space.

Lagrangian multipliers.

- The method of Lagrangian multipliers allows us to solve optimisation problems w/ equality constraints

i.e. maximise $f(x,y)$ subject to $g(x,y) = c$ ← contour line of $g(x,y)$



At the sol'n, the gradients of $f(x,y)$ and $g(x,y)$ are aligned, $\nabla f(x,y) \parallel \nabla g(x,y)$

→ The sol'n should satisfy (i) $g(x,y) = c$, (ii) $\nabla f(x,y) = -\lambda \nabla g(x,y)$

- This is equivalent to maximising the Lagrangian function $L(x,y,\lambda)$ wrt x,y,λ .

$$L(x,y,\lambda) = f(x,y) - \lambda(g(x,y) - c)$$

where λ is the Lagrange multiplier

* Note that $\nabla_{x,y} L(x,y,\lambda) = 0 \Leftrightarrow \nabla_{x,y} f(x,y) = -\lambda \nabla_{x,y} g(x,y)$; $\nabla_{\lambda} L(x,y,\lambda) = 0 \Leftrightarrow g(x,y) = c$

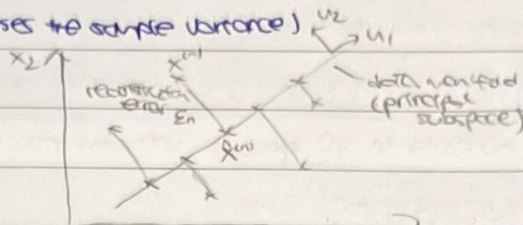
- If there are multiple restrictions $h_i(x) = 0$, $i=1,2,\dots$, the Lagrangian becomes $L(x,y,\lambda)$

$$L(x,y,\lambda) = f(x,y) - \sum_i \lambda_i h_i(x,y)$$

For Personal Use Only -bkwk2

Principal component analysis (PCA)

- PCA is a linear dimensionality method - data manifold is assumed to be linear.
- PCA finds the projection that minimises the square reconstruction error (equivalently, the projection that maximises the sample variance)



- Assume we are given a dataset $D = \{x^{(n)}\}_{n=1}^N$ w/ $x^{(n)} \in \mathbb{R}^D$, and $\sum_{n=1}^N x^{(n)} = 0$

using the orthonormal basis $\{u_i\}_{i=1}^D$, we can rewrite each dataset $x^{(n)}$ as

$$x^{(n)} = \underbrace{\sum_{i=1}^M x^{(n)T} u_i u_i}_{\text{projected vector } \hat{x}^{(n)}} + \underbrace{\sum_{i=M+1}^D x^{(n)T} u_i u_i}_{\text{reconstruction error } \epsilon_n}$$

u_i : principal comp. vectors
 $x^{(n)T} u_i$: principal comp score

the latent coordinate in new basis is $\hat{z}^{(n)} = U x^{(n)}$, where $U = \begin{bmatrix} u_1^T \\ \vdots \\ u_M^T \end{bmatrix}$ is the $M \times D$ projection matrix

- The average squared reconstruction errors is given by. (Note $\epsilon_n = \sum_{i=M+1}^D x^{(n)T} u_i u_i$)

$$\begin{aligned} \text{cost}(\{u_i\}_{i=1}^M) &= \frac{1}{N} \sum_{n=1}^N \epsilon_n^T \epsilon_n = \frac{1}{N} \sum_{n=1}^N \left(\sum_{i=M+1}^D x^{(n)T} u_i u_i^T \right) \left(\sum_{j=M+1}^D x^{(n)T} u_j u_j^T \right) \\ &= \frac{1}{N} \sum_{n=1}^N \sum_{i=M+1}^D \sum_{j=M+1}^D u_i^T x^{(n)} x^{(n)T} u_j u_i^T u_j = \frac{1}{N} \sum_{n=1}^N \sum_{i=M+1}^D u_i^T \left(\sum_{n=1}^N x^{(n)} x^{(n)T} \right) u_i \\ &= \sum_{i=M+1}^D u_i^T \left(\frac{1}{N} \sum_{n=1}^N x^{(n)} x^{(n)T} \right) u_i = \sum_{i=M+1}^D u_i^T \hat{\Sigma} u_i \end{aligned}$$

orthogonal property $u_i^T u_j = \begin{cases} 1 & i=j \\ 0 & i \neq j \end{cases}$

where $\hat{\Sigma} = \frac{1}{N} \sum_{n=1}^N x^{(n)} x^{(n)T}$ is the covariance matrix for the data.

- we use the method of Lagrange multipliers to min. the cost to ensure that $\{u_i\}_{i=1}^M$ has unit norm.

$$L(\{u_i\}_{i=1}^M, \{\lambda_i\}_{i=1}^M) = \sum_{i=M+1}^D u_i^T \hat{\Sigma} u_i + \lambda_i (1 - u_i^T u_i)$$

$$\nabla_{u_i} L(\{u_i\}_{i=1}^M, \{\lambda_i\}_{i=1}^M) = 2 \hat{\Sigma} u_i - 2 \lambda_i u_i = 0 \rightarrow \hat{\Sigma} u_i = \lambda_i u_i$$

Note $\frac{d}{d\lambda} (x^T x) = \frac{d}{d\lambda} (x^T x) = 2x$

$$\nabla_{\lambda_i} L(\{u_i\}_{i=1}^M, \{\lambda_i\}_{i=1}^M) = 1 - u_i^T u_i = 0 \rightarrow u_i^T u_i = 1$$

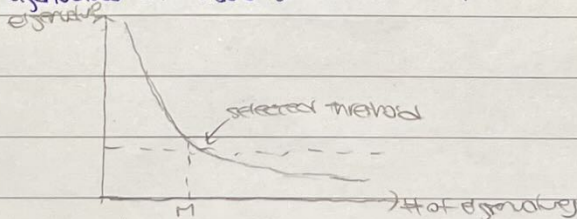
$\rightarrow (\lambda_i, u_i)$ are the eigenvalue and eigenvectors of the covariance matrix $\hat{\Sigma}$.

- we can write the objective function in terms of this sol'n

$$\text{cost}(\{u_i\}_{i=1}^M) = \sum_{i=M+1}^D u_i^T \hat{\Sigma} u_i = \sum_{i=M+1}^D u_i^T (\lambda_i u_i) = \sum_{i=M+1}^D \lambda_i (u_i^T u_i) = \sum_{i=M+1}^D \lambda_i$$

The PCA sol'n is given by the $\{u_i\}_{i=1}^M$ s.t. $u_{M+1} \dots u_D$ are eigenvectors of $\hat{\Sigma}$ w/ the $D-M$ smallest eigenvalues; $u_1 \dots u_M$ are eigenvectors of $\hat{\Sigma}$ w/ the M largest eigenvalues.

- To choose M , we sort the eigenvalues and visualise them via a plot



We choose M to be the value beyond which there is an inflection pt. The remaining eigenvalues are small w/ eigenvectors that only capture noise.

K-means clustering

clustering

- In general, two pts belonging to the same cluster are more similar/closer to each other than two pts belonging to different clusters.
- clustering is an unsupervised learning problem; no labels or rewards.
- clustering maps the dataset $D = \{x_i^{(n)}\}_{i=1}^N$ to a vector of cluster assignments $\underline{z} = \{z_1, \dots, z_N\}$

K-means clustering

we cannot use MLE to find K ,
o/w we find K to be equal to N .

- In K-means clustering, we need to specify the no. of clusters K .

- For inputs $D = \{x_i^{(n)}\}_{i=1}^N$, $x_i^{(n)} \in \mathbb{R}^D$, the K-means algorithm is as follows:

↳ From initial (random) cluster means $\underline{m}_k \in \mathbb{R}^D$ for $k=1 \dots K$, iterate the two steps

(i) Assignment step: $z_n = \arg \min_k \|x_n^{(n)} - \underline{m}_k\|_2$ for $n=1 \dots N$ [assign to closest cluster]

(ii) Update step: $\underline{m}_k = \text{mean}(x_i^{(n)} : z_n = k)$ for $k=1 \dots K$ [update the cluster means]

- K-means clustering can be interpreted as optimisation. Let $z_{n,k} = 1$ if data pt n is assigned to cluster k , and zero o/w. (Note $\sum_{k=1}^K z_{n,k} = 1$, i.e. sum of rows of $\underline{z}$ equals 1, where $z_{n,k} = (\underline{z})_{n,k}$)

$$C(\{z_{n,k}\}, \{\underline{m}_k\}) = \sum_{n=1}^N \sum_{k=1}^K z_{n,k} \|x_n^{(n)} - \underline{m}_k\|_2^2 \quad \leftarrow \text{could think as PE of a system of springs connecting } x_n^{(n)} \text{ to } \underline{m}_k.$$

K-means tries to minimise the cost function C wrt $\{z_{n,k}\}$ and $\{\underline{m}_k\}$, subject to $\sum_{k=1}^K z_{n,k} = 1$ and $z_{n,k} \in \{0,1\}$

(i) Assignment step: minimise C wrt $\{z_{n,k}\}$, holding $\{\underline{m}_k\}$ fixed.

As each data pt. is independent, minimising $C \Leftrightarrow$ minimising $\sum_{k=1}^K z_{n,k} \|x_n^{(n)} - \underline{m}_k\|_2^2$ for each row n .

We minimise $\sum_{k=1}^K z_{n,k} \|x_n^{(n)} - \underline{m}_k\|_2^2$ when $z_{n,k} = 1$ for min. $\|x_n^{(n)} - \underline{m}_k\|_2^2$ and zero o/w $\rightarrow z_n = \arg \min_k \|x_n^{(n)} - \underline{m}_k\|_2^2$

(ii) Update step: minimise C wrt $\{\underline{m}_k\}$, holding $\{z_{n,k}\}$ fixed.

As each cluster is independent, minimising $C \Leftrightarrow$ minimising $\sum_{n=1}^N \sum_{k=1}^K z_{n,k} \|x_n^{(n)} - \underline{m}_k\|_2^2$ for each cluster k

$$\frac{dC}{d\underline{m}_k} = \sum_{n=1}^N \sum_{k=1}^K z_{n,k} [-2x_n^{(n)} + 2\underline{m}_k] = 0 \rightarrow \underline{m}_k = \frac{\sum_{n=1}^N x_n^{(n)} : z_n = k}{\sum_{n=1}^N 1 : z_n = k} = \text{mean}(x_i^{(n)} : z_n = k).$$

- The cost function C is a Lyapunov function, so K-means converges. (except for edge cases)

(Finding the global optimum of the cost function C is a NP-hard problem)

- There are several issues w/ K-means clustering:

↳ The result depends on the initial cluster means $\underline{m}_k \rightarrow$ use K++ mean++ algorithm to spread out the initial centres, or repeat the algorithm multiple times and pick the best one.

↳ The algorithm returns hard assignments $\rightarrow$ use EM algorithm for soft assignments (probabilities)

↳ The algorithm assumes spherical clusters $\rightarrow$ gives strange results when the clusters are not spherical.

For Personal Use Only -bkwk2

EM algorithm

Jensen's inequality

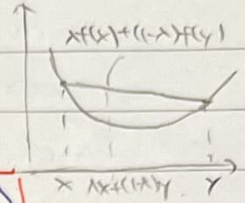
- Jensen's inequality is a general result of convexity. for a convex function f and $\lambda \in [0, 1]$

$$\lambda f(x) + (1-\lambda)f(y) \geq f(\lambda x + (1-\lambda)y)$$

we can generalise the result to expectation

$$E[f(x)] \geq f(E[x])$$

* the inequality is reversed for concave functions, so $E[f(x)] \leq f(E[x])$



Kullback-Leibler (KL) divergence.

- The KL divergence between two probability distributions $p_1(z)$ and $p_2(z)$ is defined as

$$KL(p_1(z) || p_2(z)) = \sum_z p_1(z) \log \frac{p_1(z)}{p_2(z)}$$

It is a measure of the "distance" / how similar two probability distributions are

- The KL divergence has the following properties:

(i) Non-symmetric: $KL(p_1(z) || p_2(z)) \neq KL(p_2(z) || p_1(z)) \rightarrow$ "divergence" not "distance"

(ii) Gibbs inequality: $KL(p_1(z) || p_2(z)) \geq 0$, w/ equality iff $p_1(z) = p_2(z)$. ← proof in JFT notes

Negative variational free energy $F(\theta, q(z))$

- The negative variational free energy / evidence lower bound (ELBO) is a lower bound for the likelihood $\log p(x|\theta)$, and is defined to be

$$F(\theta, q(z)) = \log p(x|\theta) - KL(q(z) || p(z|x|\theta)) = \log p(x|\theta) - \sum_z q(z) \log \frac{q(z)}{p(z|x|\theta)}$$

- Note that since KL-divergence is non-negative, we have $F(\theta, q(z)) \leq \log p(x|\theta)$, w/ equality iff $q(z) = p(z|x|\theta)$.

- To derive the negative variational free energy $F(\theta, q(z))$, consider the log-likelihood $\log p(x|\theta)$

$$\begin{aligned} \log p(x|\theta) &= \log \left[\sum_z p(x, z|\theta) \right] = \log \left[\sum_z p(z|\theta) p(x|z, \theta) \right] = \log \left[\sum_z \frac{p(z|\theta) p(x|z, \theta)}{q(z)} q(z) \right] \\ &= \log \left[E_q \left[\frac{p(z|\theta) p(x|z, \theta)}{q(z)} \right] \right] \stackrel{\text{Jensen}}{\geq} E_q \left[\log \frac{p(z|\theta) p(x|z, \theta)}{q(z)} \right] = F(\theta, q(z)) \end{aligned}$$

$$\therefore F(\theta, q(z)) = E_q[\log p(z|\theta) p(x|z, \theta)] - E_q[\log q(z)] = \sum_z q(z) \log p(z|\theta) p(x|z, \theta) - \sum_z q(z) \log q(z)$$

This form is equivalent to the form above. Noting that $p(x, z|\theta) = p(x|\theta) p(z|x|\theta) = p(z|\theta) p(x|z, \theta)$,

$$\begin{aligned} F(\theta, q(z)) &= E_q[\log p(x|\theta) p(z|x|\theta)] - E_q[\log q(z)] = \sum_z q(z) \log p(x|\theta) p(z|x|\theta) - \sum_z q(z) \log q(z) \\ &= \sum_z q(z) \log p(x|\theta) + \sum_z q(z) \log \frac{p(z|\theta)}{q(z)} \\ &= \log p(x|\theta) \sum_z q(z) - \sum_z q(z) \log \frac{q(z)}{p(z|\theta)} = \log p(x|\theta) - KL(q(z) || p(z|x|\theta)) \end{aligned}$$

+ $F(\theta, q(z)) = \log p(x|\theta) - KL(q(z) || p(z|x|\theta))$ is useful for the E-step (one term const. wrt $q(z)$)

$F(\theta, q(z)) = \sum_z q(z) \log p(z|\theta) p(x|z, \theta) - \sum_z q(z) \log q(z)$ is useful for the M-step (one term const. wrt θ)

For Personal Use Only -bkwk2

The expectation maximisation (EM) algorithm

- The EM algorithm is an iterative method to find (local) MLE of parameters θ in statistical models which involves unobserved latent variables.
- Since the negative variational free energy $F(\theta, q(z))$ is a lower bound of the log likelihood $\log p(x|\theta)$, optimising $F(\theta, q(z))$ gives a better and better lower bound for $\log p(x|\theta)$.
- The EM algorithm is as follows:

↳ From initial (random) parameters θ_0 , iterate $t=1 \dots T$ the two steps

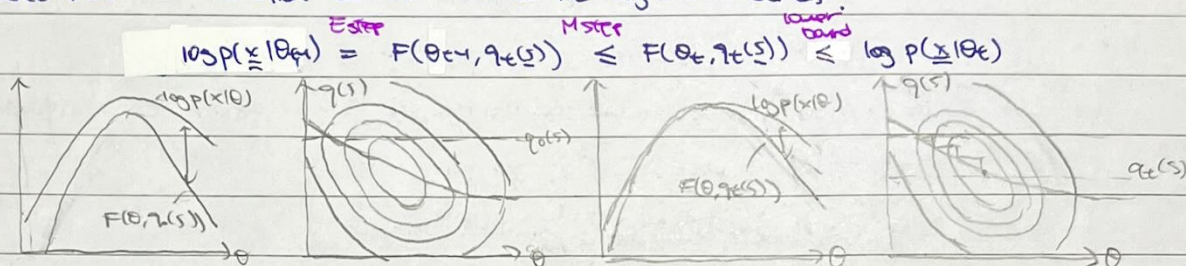
(i) E-step: For fixed θ_{t-1} , maximise lower bound $F(\theta_{t-1}, q(z))$ wrt $q(z)$

$$F(\theta_{t-1}, q(z)) = \log p(x|\theta_{t-1}) - \text{KL}(q(z) \| p(z|x, \theta_{t-1})) \rightarrow q(z) = p(z|x, \theta_{t-1})$$

(ii) M-step: For fixed $q_t(z)$, maximise the lower bound $F(\theta, q_t(z))$ wrt θ .

$$F(\theta, q_t(z)) = \sum_t q_t(z) \log p(x|z, \theta) p(z|\theta) - \sum_t q_t(z) \log q_t(z) \rightarrow \theta_t = \arg \max_{\theta} \sum_t q_t(z) \log p(x|z, \theta) p(z|\theta)$$

- Note that each iteration cannot decrease the log likelihood as



Mixture of Gaussians (MoG) generative model

- In the MoG model, for each data pt $n=1 \dots N$,

↳ the sample cluster membership distribution is given by

$$p(s_n = k | \theta) = \pi_k, \text{ where } \sum_{k=1}^K \pi_k = 1.$$

↳ the sample data value distribution given cluster membership is given by

$$p(x^{(n)} | s_n = k, \theta) = \mathcal{N}(x^{(n)} | \mu_k, \Sigma_k)$$

- We would like to first learn the parameters θ of the MoG model from data using max. likelihood, $\theta^{ML} = \arg \max_{\theta} \log p(x|\theta)$, then infer the cluster membership from the observed data $p(s_n = k | x^{(n)}, \theta^{ML})$

- The log-likelihood $\log p(x|\theta)$ is given by

$$\begin{aligned} \log p(x|\theta) &= \log \prod_{n=1}^N p(x^{(n)}|\theta) = \sum_{n=1}^N \log p(x^{(n)}|\theta) = \sum_{n=1}^N \log \sum_{k=1}^K p(x^{(n)}|s_n=k, \theta) \\ &= \sum_{n=1}^N \log \sum_{k=1}^K p(s_n=k|\theta) p(x^{(n)}|s_n=k, \theta) = \sum_{n=1}^N \log \sum_{k=1}^K \pi_k \mathcal{N}(x^{(n)} | \mu_k, \Sigma_k) \end{aligned}$$

This can be maximised using the EM algorithm or using direct gradient-based optimisation.

- Direct optimisation is generally faster, but the EM algorithm is often simpler to implement, more widely used and can lead to important generalisations.

For Personal Use Only -bkwk2

Application of the EM algorithm to the MoG model.

- Assume the data is 1D ($x \in \mathbb{R}$) for simplicity. Let the Gaussian mixture model

parameters be $\theta = \{\mu_k, \sigma_k^2, \pi_k\}_{k=1}^K$ and 1 latent variable per datapoint $z_n \in \{1, \dots, K\}$ for $n=1, \dots, N$

- Applying the EM algorithm to the simplified MoG model,

(i) E-step: Note that $p(x^{(n)} | z_n = k, \theta) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left[-\frac{1}{2\sigma_k^2}(x^{(n)} - \mu_k)^2\right]$, $p(z_n = k | \theta) = \pi_k$

$$q(z_n = k) = p(z_n = k | x^{(n)}, \theta) \propto p(x^{(n)} | z_n = k, \theta) p(z_n = k | \theta) = \frac{\pi_k}{\sum_{k=1}^K \pi_k} \exp\left[-\frac{1}{2\sigma_k^2}(x^{(n)} - \mu_k)^2\right] = r_{n,k}$$

$$\therefore q(z_n = k) = \frac{r_{n,k}}{\sum_{k=1}^K r_{n,k}} = r_{n,k} \quad \text{("} r_{n,k} \rightarrow 1 \text{: high responsibility, } r_{n,k} \rightarrow 0 \text{: low responsibility, "responsibility" that component } k \text{ takes for datapoint } n \text{)}$$

(ii) M-step: $F(\theta, q(z)) = \sum_{n=1}^N q(z_n) \log p(z_n | \theta) p(x^{(n)} | z_n, \theta) - \sum_{n=1}^N q(z_n) \log q(z_n)$

$$= \sum_{n=1}^N \sum_{k=1}^K q(z_n = k) \left[\log \pi_k - \frac{1}{2} \log \sigma_k^2 - \frac{1}{2\sigma_k^2} (x^{(n)} - \mu_k)^2 \right] + \text{const.}$$

$$\therefore \frac{\partial F}{\partial \mu_j} = \sum_{n=1}^N q(z_n = j) \frac{x^{(n)} - \mu_j}{\sigma_j^2} = 0 \rightarrow \mu_j = \frac{\sum_{n=1}^N q(z_n = j) x^{(n)}}{\sum_{n=1}^N q(z_n = j)}$$

$$\frac{\partial F}{\partial \sigma_j^2} = \sum_{n=1}^N q(z_n = j) \left[-\frac{1}{2\sigma_j^2} + \frac{(x^{(n)} - \mu_j)^2}{2\sigma_j^4} \right] = 0 \rightarrow \sigma_j^2 = \frac{\sum_{n=1}^N q(z_n = j) (x^{(n)} - \mu_j)^2}{\sum_{n=1}^N q(z_n = j)}$$

$$\frac{\partial (F + \lambda(1 - \sum_{k=1}^K \pi_k))}{\partial \pi_j} = \sum_{n=1}^N \frac{q(z_n = j)}{\pi_j} - \lambda = 0 \rightarrow \pi_j = \frac{1}{N} \sum_{n=1}^N q(z_n = j) \quad \begin{matrix} \sum_{j=1}^K q(z_n = j) = 1 \\ \therefore \sum_{n=1}^N \sum_{j=1}^K q(z_n = j) = N \end{matrix}$$

- In summary, the EM algorithm for the MoG model is as follows:

→ From initial (random) parameters $\theta = \{\mu_k, \sigma_k^2, \pi_k\}_{k=1}^K$, iterate the two steps:

(i) E-step: $r_{n,k} = p(z_n = k | x^{(n)}, \theta)$ for $n=1, \dots, N$

(ii) M-step: $\theta = \arg \max_{\theta} \sum_{n=1}^N \sum_{k=1}^K r_{n,k} p(x^{(n)} | z_n = k, \theta)$

- Applying the EM algorithm to the MoG model is soft K-means clustering w/ non-axis aligned, non-equally weighted clusters.

- Applying the EM algorithm to the MoG model has several drawbacks:

→ MoG clusters still have simple shapes → one cluster can have many components / use complex models

→ ML approach could overfit → use Bayesian approach

→ coordinate descent is often slow to converge → joint optimisation of $F(\theta, q(z))$ or direct optimisation of the log-likelihood $\log p(x | \theta)$

→ The free energy may diverge (as it is not bounded above) → "robust problem" when one of the datapoints take the entire cluster.

sequence modelling

Markov models

sequence modelling.

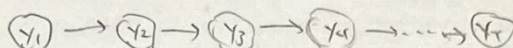
- sequence modelling can be used to:

- ↳ predict future item in a sequence $p(y_t | y_{1:t-1})$ [eg: FTSE 100 trends]
- ↳ remove noise from a sequence $p(y_{1:t} | y_{1:t})$ [eg: noise removal]
- ↳ predict one sequence from another $p(y_{1:t} | y_{1:t})$ [eg: translation]
- ↳ discover underlying latent variables $p(x_{1:t} | y_{1:t})$ [eg: sentiment analysis]

Markov models.

- A Markov model has the Markov (memoryless) property.

(i) 1st order Markov: $y_{t+1} \perp y_{1:t-1} | y_t$ $[p(y_{1:t}) = p(y_1)p(y_2|y_1)p(y_3|y_2)\dots p(y_t|y_{t-1})]$



(ii) 2nd order Markov: $y_{t+1} \perp y_{1:t-2} | y_t, y_{t-1}$ $[p(y_{1:t}) = p(y_1)p(y_2|y_1)p(y_3|y_1, y_2)\dots p(y_t|y_{t-2}, y_{t-1})]$



- Markov models for discrete data — N-gram models

(i) 1st order Markov (Bi-gram): $y_t \in \{1 \dots K\}$, $p(y_1 = k) = \pi_k^0$, $p(y_t = k | y_{t-1} = l) = T_{kl}$

(ii) 2nd order Markov (Tri-gram): $y_t \in \{1 \dots K\}$, $p(y_1 = k) = \pi_k^0$, $p(y_t = k | y_{t-1} = l, y_{t-2} = m) = T_{klm}$

- For the bi-gram model, the marginal distribution for the second state $p(y_2 = k)$ is given by

$$p(y_2 = k) = \sum_{l=1}^K p(y_2 = k | y_1 = l) p(y_1 = l) = \sum_{l=1}^K T_{kl} \pi_l^0 = [\mathbf{T} \boldsymbol{\pi}^0]_k \rightarrow p(y_2) = \mathbf{T} \boldsymbol{\pi}^0$$

We can find the stationary distribution of the Markov chain by finding the eigenvectors of the transition matrix $\mathbf{T}$ w/ eigenvalue equal to one, since

$$p(y_t = k) = \sum_{l=1}^K p(y_t = k | y_{t-1} = l) p(y_{t-1} = l) \rightarrow \pi_k^0 = \sum_{l=1}^K T_{kl} \pi_l^0 \rightarrow \boldsymbol{\pi}^0 = \mathbf{T} \boldsymbol{\pi}^0 \quad (\lambda=1)$$

* We can find the max. likelihood transition matrix from an observed sequence by considering the proportions of transitions from state i to state j (out of all transitions from state i)

- Markov models for continuous data — Auto-regressive (AR) models

(i) 1st order Markov (AR(1)): $y_t \in \mathbb{R}^D$, $p(y_1) = N(y_1 | \mu_0, \Sigma_0)$, $p(y_t | y_{t-1}) = N(y_t | \Delta y_{t-1}, \Sigma)$ $y_t = \Delta y_{t-1} + \sum_{i=1}^D \epsilon_i e_i$, $\epsilon_i \sim N(0, 1)$

(ii) 2nd order Markov (AR(2)): $y_t \in \mathbb{R}^D$, $p(y_1) = N(y_1 | \mu_0, \Sigma_0)$, $p(y_t | y_{t-1}, y_{t-2}) = N(y_t | \Delta_1 y_{t-1} + \Delta_2 y_{t-2}, \Sigma)$ $y_t = \Delta_1 y_{t-1} + \Delta_2 y_{t-2} + \sum_{i=1}^D \epsilon_i e_i$, $\epsilon_i \sim N(0, 1)$

- For AR(1) and $y_t \in \mathbb{R}^D$, $p(y_t | y_{t-1}) = N(y_t | \lambda y_{t-1}, \sigma^2) \hookrightarrow y_t = \lambda y_{t-1} + \sigma \epsilon_t$, $\epsilon_t \sim N(0, 1)$

Since $p(y_{1:t})$ is Gaussian, and the marginal of a Gaussian is Gaussian, the stationary distribution $p(y_\infty)$ is Gaussian

$$\hookrightarrow \text{mean: } E[y_t] = \lambda E[y_{t-1}] + \sigma^2 E[\epsilon_t] = \lambda E[y_{t-1}] \rightarrow \mu_t = \lambda \mu_{t-1}$$

$$\text{As } t \rightarrow \infty, \mu_{t-1} = \lambda \mu_{t-2} \rightarrow \mu_{\infty} = 0$$

$$\hookrightarrow \text{variance: } E[y_t^2] = \lambda^2 E[y_{t-1}^2] + 2\lambda\sigma E[y_{t-1}\epsilon_t] + \sigma^2 E[\epsilon_t^2] = \lambda^2 E[y_{t-1}^2] + 2\lambda\sigma E[y_{t-1}] E[\epsilon_t] + \sigma^2 \rightarrow \sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + \sigma^2$$

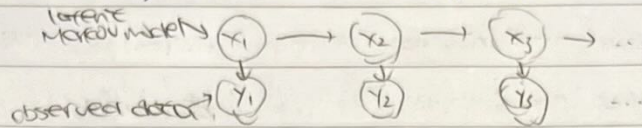
$$\text{As } t \rightarrow \infty, \sigma_{t-1}^2 = \lambda \sigma_{t-2}^2 + \sigma^2 \rightarrow \sigma_{\infty}^2 = \frac{\sigma^2}{1-\lambda^2}$$

+ We always set λ s.t. its eigenvalue has absolute value less than 1 $\rightarrow \sigma_{\infty}^2$ does not blow up

Hidden Markov models

Hidden Markov models

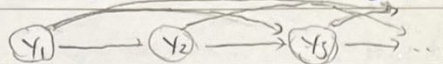
- In hidden Markov models, the real (observed) data y_t depend on latent (unobserved) variables x_t .



The joint distribution over all the latent variables $x_{1:T}$ and real data $y_{1:T}$ is given by

$$p(y_{1:T}, x_{1:T}) = \prod_{t=1}^T p(x_t | x_{t-1}) p(y_t | x_t), \quad \text{where } p(x_1 | x_0) = p(x_1)$$

We can marginalise the latents and get a fully-connected graph (not Markovian)



- Two prevalent examples include:

↳ Hidden Markov model (HMM): discrete x , discrete/continuous y

↳ Linear Gaussian state space model (LGSSM): continuous x , continuous y .

Hidden Markov model — discrete hidden state (HMM)

- HMMs have discrete hidden states: $x_t \in \{1, \dots, K\}$, $p(x_t = k | x_{t-1} = l) = T_{kl}$.

- The observed state can be continuous or discrete

↳ continuous observed state: $p(y_t | x_t = k) = N(y_t | \mu_k, \Sigma_k)$.

↳ discrete observed state: $p(y_t = l | x_t = k) = S_{lk}$, where S is the emission matrix.

- For discrete hidden state, continuous observed state,

$$p(y_1) = \sum_{k=1}^K p(y_1 | x_1 = k) p(x_1 = k) = \sum_{k=1}^K N(y_1 | \mu_k, \Sigma_k) T_{k1}$$

$$p(y_{1:T}) = \sum_{k=1}^K p(y_{1:T} | x_{1:T} = k) p(x_{1:T} = k) = \sum_{k=1}^K N(y_{1:T} | \mu_k, \Sigma_k) T_{k1}$$

← Mixture of Gaussians

→ we can interpret them as a MoG model w/ dynamic cluster assignments.

Hidden Markov model — continuous hidden state (LGSSM)

- LGSSMs have continuous hidden states: $x_t \in \mathbb{R}^K$, $p(x_t | x_{t-1}) = N(x_t | A x_{t-1}, Q)$

- The observed state is continuous

↳ continuous observed state: $y_t \in \mathbb{R}^D$, $p(y_t | x_t) = N(y_t | C x_t, R)$

← $D \times K$ matrix

For Personal Use Only -bkwk2

Varities of inference

The four distributional estimates are as follows:

		infer at the state vs sequence? (single state) marginal	(sequence) joint
filter/online yes	filter/online	$P(x_t y_{1:t})$	$P(x_{1:t} y_{1:t})$
	smoother/offline	$P(x_t y_{1:T})$	$P(x_{1:t} y_{1:T})$

A diff. algorithm is req. for each of the distributional estimates $\rightarrow$ JF8 focuses on $p(x_t | y_{1:t})$

The two point estimates are as follows:

$$\rightarrow x_t^* = \underset{x_t \in \mathcal{X}}{\operatorname{argmax}} p(x_t | y_{1:t}) \leftarrow \text{"bfc-optimal"} \quad \rightarrow x_{1:t}^* = \underset{x_{1:t} \in \mathcal{X}_{1:t}}{\operatorname{argmax}} p(x_{1:t} | y_{1:t}) \leftarrow \text{"black-optimal"}$$

The estimates $x_{1:t}^*$ and x_t^* are the same for the LGSSM (but not discrete hidden state HMM) $\rightarrow$

For LGSSM, $p(x_{1:t} | y_{1:t}) = N(x_{1:t} | M_{1:t}, \Sigma_{1:t})$ [Gaussian since $p(x_{1:t}, y_{1:t})$ is Gaussian]

$$\rightarrow x_t^* = \underset{x_t \in \mathcal{X}}{\operatorname{argmax}} p(x_t | y_{1:t}) = M_t \rightarrow x_{1:t}^* = M_{1:t}$$

shown with counterexample

$$\rightarrow x_{1:t}^* = \underset{x_{1:t} \in \mathcal{X}_{1:t}}{\operatorname{argmax}} p(x_{1:t} | y_{1:t}) = M_{1:t}$$

Kalman filter

The Kalman filter is an iterative algorithm to compute $p(x_t | y_{1:t})$ for the LGSSM.

(i) start w/ $p(x_1 | y_{1:t-1}) = N(x_1 | \bar{M}_1^x, \bar{\Sigma}_1^x)$ use $x_t = \underline{A}x_{t-1} + \underline{Q}^{1/2}\xi_t, \xi_t \sim N(0, I)$

(ii) Diffuse via dynamics: $p(x_t | y_{1:t}) = \int p(x_t | x_{t-1}) p(x_{t-1} | y_{1:t-1}) dx_{t-1}$

$$p(x_t | y_{1:t}) = \int N(x_t | \underline{A}x_{t-1}, \underline{Q}) N(x_{t-1} | \bar{M}_{t-1}^x, \bar{\Sigma}_{t-1}^x) dx_{t-1} = N(x_t | \bar{M}_t^x, \bar{\Sigma}_t^x)$$

where $\bar{M}_t^x = \underline{A} \bar{M}_{t-1}^x, \bar{\Sigma}_t^x = \underline{A} \bar{\Sigma}_{t-1}^x \underline{A}^T + \underline{Q}$

(iii) combine w/ likelihood: $p(x_t | y_{1:t}) \propto p(x_t | y_{1:t-1}) p(y_t | x_t)$

$$\therefore p(x_t | y_{1:t}) = N(x_t | \bar{M}_t^x, \bar{\Sigma}_t^x)$$

where $\bar{M}_t^x = \bar{M}_t^x + K_t(y_t - \underline{C}\bar{M}_t^x), \bar{\Sigma}_t^x = \bar{\Sigma}_t^x - K_t \underline{C} \bar{\Sigma}_t^x, K_t = \bar{\Sigma}_t^x \underline{C}^T (\underline{C} \bar{\Sigma}_t^x \underline{C}^T + R)^{-1}$

Forward algorithm

Markov property allows us to use dynamic programming / belief propagation for efficient computation.

The forward algorithm is an iterative algorithm to compute $p(x_t | y_{1:t})$ for discrete hidden state HMM.

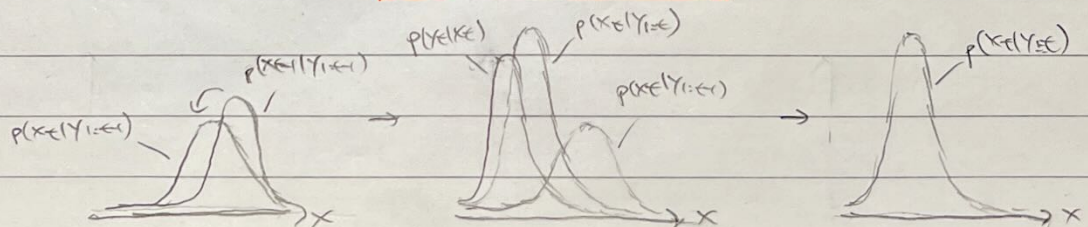
(i) start w/ $p(x_1 = k | y_{1:t-1}) = \bar{p}_{t-1}^k(k)$

(ii) Diffuse via dynamics: $p(x_t = k | y_{1:t-1}) = \sum_{l=1}^K p(x_t = k | x_{t-1} = l) p(x_{t-1} = l | y_{1:t-1})$

$$\bar{p}_t^k(k) = \sum_{l=1}^K T_{kl} \bar{p}_{t-1}^l(l)$$

(iii) combine w/ likelihood: $p(x_t = k | y_{1:t}) \propto p(x_t = k | y_{1:t-1}) p(y_t | x_t = k)$

$$\bar{p}_t^k(k) \propto \bar{p}_t^k(k) p(y_t | x_t = k)$$



For Personal Use Only -bkwk2

Derivation of the general filtering equations.

$$① \boxed{p(x_t | y_{1:t-1}) = \int p(x_t | x_{t-1}) p(x_{t-1} | y_{1:t-1}) dx_{t-1}}$$

← replace integrals w/ sums for the discrete case

$$\hookrightarrow p(x_t | y_{1:t-1}) = \int p(x_t, x_{t-1} | y_{1:t-1}) dx_{t-1} = \int p(x_t | x_{t-1}, y_{1:t-1}) p(x_{t-1} | y_{1:t-1}) dx_{t-1}$$

$$= \int p(x_t | x_{t-1}) p(x_{t-1} | y_{1:t-1}) dx_{t-1}$$

cond. indep. from model

$$② \boxed{p(x_t | y_{1:t}) \propto p(y_t | x_t) p(x_t | y_{1:t-1})}$$

cond. indep. from model

$$\hookrightarrow p(x_t | y_{1:t}) = \frac{p(y_t | x_t, y_{1:t-1}) p(x_t | y_{1:t-1})}{p(y_t | y_{1:t-1})} = \frac{p(y_t | x_t) p(x_t | y_{1:t-1})}{p(y_t | y_{1:t-1})}$$

$$\propto p(y_t | x_t) p(x_t | y_{1:t-1})$$

Computing the likelihood.

- The likelihood $p(y_{1:T})$ is given by

$$\boxed{p(y_{1:T}) \prod_{t=1}^T p(y_t | y_{1:t-1})}$$

where the term $p(y_t | y_{1:t-1})$ is the normalisation const. for computing $p(x_t | y_{1:t})$

→ we simply store $p(y_t | y_{1:t-1})$ for each t and find the product to get the likelihood $p(y_{1:T})$